



AgentIA Engineering

September 25, 2026

The problem, in one sentence

AgentIA's failures are **procedural**, not capability gaps: the model *knows* the rule (use `jakarta.*`; don't inject repositories into controllers) but forgets to apply it in a given session.

Two failure classes

- **Capability gap** — the model doesn't know the rule. No reminder helps.
- **Application gap** — the model knows but doesn't apply. A persistent, triggered reminder helps.

SkillOpt targets the second. It improves *reliability within scope*, not coverage.

What a skill actually is

Not a cheat sheet. A **procedural policy** with three parts:

- ① **Trigger** — `when_to_apply`
- ② **Procedure** — ordered rules
- ③ **Granularity** — task-level or event-driven

Example (from CODESKILL's Java/Maven set)

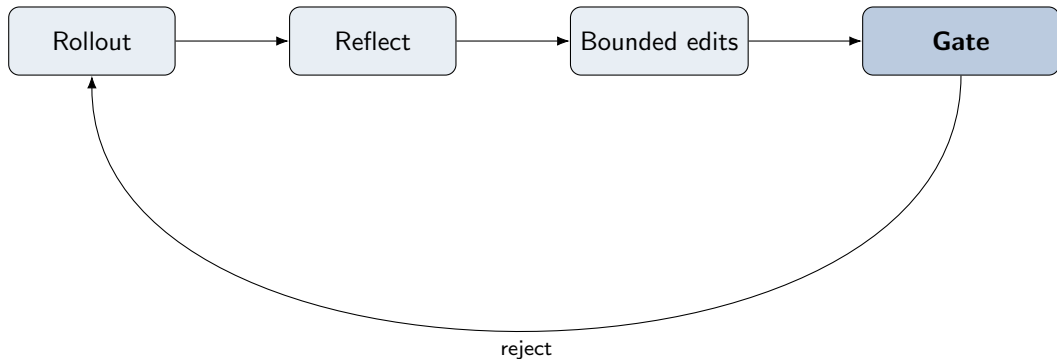
When. A Maven build fails with a test-jar dependency-resolution error.

Rules. Inspect the POM scope. Check the producing module for a plugin execution. Run tests or the test-jar goal explicitly. Do not retry with tests skipped. Run a full install. Verify presence before retrying.

At deployment: prepended to the generation prompt. Model stays frozen.

What SkillOpt is

Treat the skill document as a **trainable parameter**. Build a training loop around it — in text space, with the discipline of weight-space optimization.



Accepted only if the candidate *strictly* improves a held-out score. Ties rejected.

The four controls

Bounded edit budget $\leq L_t$ edits per step (default 4, cosine decay). Prevents unbounded rewriting.

Held-out validation gate Accept iff selection score strictly improves. Makes the loop auditable and non-degrading.

Rejected-edit buffer Failed edits fed back as negative evidence. Avoids repeated proposals.

Epoch-wise slow/meta update Longitudinal guidance written to a protected section. Catches drift.

The loop

Inner loop (per step)

rollout on training set → split success/failure → optimizer proposes edits → merge, dedupe, rank → clip to L_t → apply → **gate**.

Outer loop (per epoch, 4 total)

sample 20 tasks → re-run under previous and current skill → categorize → write longitudinal guidance to protected section → same gate.

Output: best_skill.md, 300–2,000 tokens, inspectable, zero inference-time cost.

The evidence

- Best or tied-best on **52 of 52** (model, benchmark, harness) cells.
- GPT-5.5 direct chat: +23.5 pts over no-skill; Codex: +24.8; Claude Code: +19.1.
- Beats an *oracle* per-cell baseline by +5.4 pts.
- **Edit economy:** 1–4 accepted edits produce +29 to +39 pts on procedural benchmarks.
- **Cross-harness transfer:** Codex-trained skill transfers to Claude Code at +59.7 over baseline.

The ablation that matters most

Component	SpreadsheetBench	Δ
Full SkillOpt	77.5	—
Without meta skill	75.7	−1.8
Without meta + slow update	55.0	−22.5

The epoch-level longitudinal view is **load-bearing**, not decorative. Do not ship without it.

- **No code benchmark.** None of the six benchmarks is code. Mechanism is domain-agnostic; numbers do not transfer.
- **Requires an automated verifier.** The paper is explicit. `mvn test -o` satisfies this.
- **One document, not a bank.** Bank curation (SkillBrew, Ratchet) is deferred.
- **No weight changes.** No training infrastructure. That is a feature.

The staged plan

Spec 1 — inject and measure (1–2 weeks)

- ① **Task 0:** query `studio.db` for historical pass/fail and error classification. Triage domains.
- ② Hand-write seed skills for surviving domains.
- ③ Add `skills` table, `skill_injection_node`, injection logging, re-injection on repair.
- ④ Four-condition A/B pilot (≥ 15 sessions per cell).

Spec 2 — optimize (4 weeks, conditional)

Full SkillOpt loop. Different-provider optimizer via LLMFactory. Only if Spec 1 shows signal.

The pilot design

Condition	Prompt addition	Purpose
no_skills	none	Baseline
skills	active skill docs	Treatment
sham	~1.5K tokens of noise	Prompt-length control
skills_plus_fixer	skills + fixer	Interaction

Primary metric: `mvn test -o` pass rate.

Secondary: reviewer tag (as-is / minor / rewrite).

Random assignment within the same agent version and window.

Decision points

- ① **After Task 0.** Which domains survive triage? Which go to deterministic fixers?
- ② **After Spec 1.** Does injection move the primary metric? If no, stop. If yes, build Spec 2.
- ③ **Before Spec 2.** Spec 2 vs. coverage expansion (Spring Security). Equal footing. Not momentum.

The arithmetic worth stating

Reliability on 30–40% of the requisition cannot reach a target defined against 100% while 60–70% is uncovered. Reliability is necessary infrastructure for coverage expansion — not a substitute.

- Li et al. *SkillOpt: A controllable text-space optimizer for agent skills*. arXiv:2605.23904 (2026).
- Li et al. *CODESKILL: Learning self-evolving skills for coding agents*. arXiv:2605.25430 (2026).
- Hu et al. *SkillBrew: Multi-objective curation of skill banks for LLM agents*. arXiv:2605.29440 (2026).
- Eliseeva et al. *EnvBench*. arXiv:2503.14443 (2025).